

BUG-001 · Unstructured PDF conversion failed because PDF extras were missing

Field	Detail
Issue	<code>unstructured_test.py</code> could not load the PDF partitioner and raised <code>ModuleNotFoundError: No module named 'unstructured_inference'</code> .
Found Date	2026-07-14
Status	Closed
Severity	HIGH
File	<code>unstructured_test.py</code> ; Python environment
Description	The base package/import was initially absent, and a later run with incomplete dependencies reached <code>unstructured.partition.pdf_image.pdfminer_processing</code> before failing to import <code>unstructured_inference</code> .
Root Cause	PDF support depends on optional inference/OCR packages not supplied by a minimal Unstructured installation.
Solution	Install <code>unstructured[pdf]</code> in the Python 3.12 environment and verify the target PDF through PowerShell; see ADR-002 and Research topic 2.
Date Resolved	2026-07-14

BUG-002 · Marker installation failed on Python 3.14 at the regex dependency

Field	Detail
Issue	Installing <code>marker-pdf[full]</code> failed while building <code>regex==2024.11.6</code> .
Found Date	2026-07-14
Status	Closed
Severity	HIGH
File	Python environment; <code>marker_test.py</code> blocked
Description	The installer could not find a compatible wheel and attempted a native source build that required unavailable Microsoft C++ build tooling.
Root Cause	Python 3.14 was newer than the supported Windows wheel set for the pinned dependency.
Solution	Create a Python 3.12 environment and install <code>marker-pdf[full]</code> , which resolved compatible wheels; standardized in ADR-002.
Date Resolved	2026-07-14

BUG-003 · Unstructured output directory was absent before the first write

Field	Detail
Issue	<code>unstructured_test.py</code> targeted <code>unstructured_results/*.md</code> while the directory did not exist.
Found Date	2026-07-14
Status	Closed
Severity	MEDIUM
File	<code>unstructured_test.py</code> ; <code>unstructured_results/</code>

Description	Python's <code>open()</code> creates a file but not its missing parent directory, so the planned write would fail with <code>FileNotFoundError</code> .
Root Cause	The script assumes result directories are provisioned externally.
Solution	Initially create <code>unstructured_results/</code> manually. ADR-004 later removed the underlying assumption: all three scripts now create their output directories and mirrored subdirectories automatically.
Date Resolved	2026-07-14

BUG-004 · Git Bash reported false native-process failures for Unstructured

Field	Detail
Issue	Invoking the Windows Python 3.12 interpreter through Git Bash produced exit 139/127 and a reported segmentation fault with no useful log.
Found Date	2026-07-14
Status	Closed
Severity	MEDIUM
File	Execution environment; <code>unstructured_test.py</code>
Description	Basic Python worked, but heavy Unstructured/native DLL imports behaved inconsistently through Git Bash. The same import and full script worked through PowerShell and generated both outputs.
Root Cause	Shell/process interoperability around the Windows native Python and its heavy DLL stack, not a reproducible application-code crash.
Solution	Use PowerShell for Windows execution and verification; see ADR-003.
Date Resolved	2026-07-14

BUG-005 · Shared installation selected a Python-incompatible Numba release

Field	Detail
Issue	Installing the unpinned combined requirements.txt selected numba==0.53.1, which refuses Python 3.12.
Found Date	2026-07-15
Status	Closed
Severity	HIGH
File	Temporary requirements.txt; shared Python environment
Description	uv pip install -r requirements.txt failed while building Numba because that release supports Python 3.6 through 3.9.
Root Cause	The unconstrained dependency resolution selected an obsolete transitive Numba version for Unstructured.
Solution	Abandon the combined environment and install each pinned loader in its own Python 3.12 environment under ADR-005.
Date Resolved	2026-07-15

BUG-006 · Marker and Unstructured have incompatible Pillow constraints

Field	Detail
Issue	Marker PDF 1.10.2 and unstructured[pdf] 0.24.1 could not be resolved in one environment.
Found Date	2026-07-15
Status	Closed
Severity	HIGH
File	Temporary requirements.txt; shared Python environment
Description	Unstructured's pi-heif dependency required Pillow 11.1 or newer,

	while Marker required Pillow below 11.
Root Cause	The selected upstream releases declare mutually exclusive version ranges for Pillow.
Solution	Use the three loader-specific environments defined by ADR-005; remove the unsatisfiable shared requirements and lockfile.
Date Resolved	2026-07-15

BUG-007 · Shared environment contained incomplete Torch metadata

Field	Detail
Issue	uv could not read the installed Torch distribution because its METADATA file was missing.
Found Date	2026-07-15
Status	Closed
Severity	HIGH
File	Temporary .venv; Torch installation
Description	Repeated installs failed while reading torch-2.13.0.dist-info/METADATA, indicating a corrupted or incomplete package state.
Root Cause	The experimental shared environment accumulated a partial Torch installation while dependencies were repeatedly added and re-resolved.
Solution	Delete the shared environment and recreate isolated environments from scratch (ADR-005).
Date Resolved	2026-07-15

BUG-008 · Unrelated marker package shadowed Marker PDF

Field	Detail
Issue	marker_test.py imported an unrelated distribution and failed with ImportError: cannot import name 'CONFIG_DIR' from 'marker.utils'.
Found Date	2026-07-15
Status	Closed
Severity	HIGH
File	Python environment; codes/marker_test.py blocked

Description	The installed package exposed <code>marker.lms.Markus</code> , not Datalab's <code>marker.converters.pdf.PdfConverter</code> implementation expected by the script.
Root Cause	The distribution named <code>marker</code> was installed instead of <code>marker-pdf</code> , creating a namespace collision.
Solution	Create <code>.venv-marker</code> and install <code>marker-pdf==1.10.2</code> ; do not install the unrelated <code>marker</code> distribution.
Date Resolved	2026-07-15

BUG-009 · Docling can report batch success after page preprocessing failures

Field	Detail
Issue	Docling emitted <code>Stage preprocess failed ... std::bad_alloc</code> for individual pages, but the runner still counted the containing document as converted.
Found Date	2026-07-15
Status	Open
Severity	HIGH
File	<code>codes/docling_test.py</code> ; Docling preprocessing pipeline
Description	Native page preprocessing ran on a CPU-only Torch build and repeatedly failed to allocate memory, often after the first ten zero-based page indexes. Docling returned partial document objects, so the script wrote Markdown and printed <code>6 converted, 0 failed</code> .
Root Cause	The native preprocessing stage can fail below the document-level exception boundary; the runner validates only whether <code>converter.convert()</code> returns, not page completeness. CPU buffering, a native allocation leak, or OCR pressure remained suspected rather than proven.
Solution	Not yet implemented. Candidate diagnostics are to disable OCR for text PDFs, reduce OCR/layout/table batch sizes and queue depth, preserve a verified CUDA Torch pin, and add page-level

	completeness checks.
Date Resolved	—

BUG-010 · Marker Markdown contains broken image links and mojibake

Field	Detail
Issue	The six-file Marker run produced Markdown with 131 image references but no corresponding image assets; the RAG chapter also contains at least 26 mojibake sequences.
Found Date	2026-07-15
Status	Open
Severity	HIGH
File	codes/marker_test.py; marker_results/; Marker-PDF Report.md
Description	Every referenced logo, diagram, map, engineering graphic, and page image is broken in the delivered result tree. Corrupted punctuation and symbols remain in the RAG output despite UTF-8 writes.
Root Cause	The runner writes only Markdown from <code>text_from_rendered()</code> and does not export or validate image assets. The exact layer introducing mojibake has not been isolated.
Solution	Not yet implemented. Export assets to stable relative paths, validate links, and scan/normalize text encoding before accepting a conversion.
Date Resolved	—

BUG-011 · Marker can silently corrupt structural completeness despite high word retention

Field	Detail
Issue	Page-level comparison found incorrect table row associations, missing navigation data, interrupted lists, and reordered, duplicated, or omitted content even though aggregate word recovery remained high.
Found Date	2026-07-16
Status	Open
Severity	HIGH
File	codes/marker_test.py; local marker_results/; Marker-PDF Report.md
Description	In the first RFT continuation volume, the page 5 contact table split and merged organizations into the wrong rows, the repeated RFT title was omitted, page 204 table-of-contents numbers disappeared, and running headers interrupted lists from about page 220. In the second continuation volume, a dense table around pages 61–62 was reordered, section 2.1.2 was duplicated, and later content was missing.
Root Cause	Marker’s layout classification and Markdown serialization do not reliably preserve two-dimensional row relationships, page furniture, pagination, or cross-page sequence. The exact trigger for the pages 61–62 loss has not been isolated.
Solution	Not yet implemented. Preserve page/block metadata, enable targeted source-to-output validation at table and page boundaries, detect duplicate section labels and missing TOC pagination, and quarantine structurally complex pages before chunking or retrieval.
Date Resolved	—

BUG-012 · Marker flattens nested Markdown list pointers to one indentation level

Field	Detail
Issue	Marker's Markdown output represents multi-level list pointers (bullet-symbol sub-points, parenthetical roman-numeral sub-points) as flat - items at the same indentation as their parent pointer, discarding the original nesting depth.
Found Date	2026-07-17
Status	Open
Severity	MEDIUM
File	codes/marker_test.py; marker_results/; surfaced while inspecting chunking/ingest.py reports
Description	Found by manually comparing chunk reports against the source RFT documents while iterating on the Markdown splitter: sub-points such as "• the requirements..." and "(i) the requirements..." are emitted as top-level - list items, so no indentation signal distinguishes a pointer from its sub-pointers.
Root Cause	Not isolated. Marker's list-block serialization does not appear to preserve original nesting depth for bullet- and roman-numeral-style sub-points.
Solution	Not yet implemented. A regex-based normalization pass over Marker's Markdown output is planned, to reconstruct indentation from marker glyphs (for example •, parenthetical numerals) before chunking.
Date Resolved	—

BUG-013 · preprocessing() scoped alphabetic/roman marker families globally, chaining unrelated enumerations

Field	Detail
Issue	Alphabetic ((a)) and roman ((i)) marker sequences were keyed by a bare "alpha"/"roman" family string, so unrelated enumerations anywhere in the document chained into one run whenever their values happened to continue in sequence, splicing unrelated clauses together (e.g. a Section 6 roman sub-list glued after clause 6.20).
Found Date	2026-07-20
Status	Closed
Severity	HIGH
File	chunking/ingest.py (_marker_sequences(), _marker_family_value())
Description	Found by diffing preprocessing() output against the full <code>marker_results/expected-output.md</code> ground truth, which the two earlier toy examples were too small to expose. A related second defect applied both an outer heading-promoted edit and a fully-nested inner edit over overlapping spans, scrambling the text; resolved in the same pass by keeping only the outermost of any overlapping edits.
Root Cause	Alphabetic and roman families carried no positional/section context, so distinct enumerations shared a single family key and were treated as one continuing run.
Solution	Reset any in-progress alphabetic/roman run at every heading or numeric-marker boundary, and drop fully-nested/overlapping edits so only the outermost is applied.
Date Resolved	2026-07-20

BUG-014 · preprocessing() unconditional forward-merge concatenated unrelated blocks

Field	Detail
Issue	The clause-body rejoining step merged every following blank-line-delimited block into a clause until the next marker, regardless of content, gluing unrelated material together — e.g. a run of separate glossary definitions ("DELAY EVENT", "DISPUTE", "DRAWINGS", "EAD", ...) collapsed onto a single line.
Found Date	2026-07-20
Status	Closed
Severity	HIGH
File	<code>chunking/ingest.py (merge_forward(), apply_heading_branch(), apply_plain_branch())</code>
Description	Exposed by diffing <code>preprocessing()</code> output against the full <code>marker_results/expected-output.md</code> ground truth. The "always merge a clause's trailing fragment forward to the next marker" behaviour — a simplification agreed after the two earlier toy examples — was corrupting real output wherever consecutive non-clause blocks followed a marker.
Root Cause	<code>merge_forward()</code> had no stopping condition, so it bridged paragraph breaks past the end of the clause's actual body.
Solution	Add a sentence-boundary heuristic: only bridge the next fragment while the accumulated text does not yet end in terminal punctuation (<code>/!/?</code>). Also extended the merge behaviour, previously wired only into the heading branch, to the plain branch.
Date Resolved	2026-07-20

BUG-015 · temp_split() nested-list boundary helper silently dropped content between chunks

Field	Detail
Issue	While relaxing the list-with-lead-in rules to accept nested lists, <code>top_level_item_boundaries()</code> omitted the starting cursor when that cursor sat mid-way through a nested run, so text between it and the next true top-level pointer vanished from every emitted chunk.
Found Date	2026-07-20
Status	Closed
Severity	HIGH
File	<code>chunking/ingest.py</code> (<code>top_level_item_boundaries()</code> , <code>temp_split()</code>)
Description	Surfaced by a full round-trip check (every emitted chunk must reappear in the preprocessed source with nothing left over) run across all four local <code>marker_results/</code> documents: 13 non-blank gaps appeared where a previous cut had left the cursor inside a nested (i)/(ii)/(iii) sub-list, and the returned boundary list started at the first top-level pointer instead of at the cursor.
Root Cause	The new oversized-list fallback computed split points only at top-level list pointers and never forced the actual start position in as the first boundary, so any content before the first top-level pointer in the window was skipped.
Solution	Always insert the start position as the first boundary in <code>top_level_item_boundaries()</code> . Re-verified zero non-blank gaps and zero unconsumed tail across all four documents.
Date Resolved	2026-07-20

BUG-016 · Concurrent Marker converters abort the shared llama-server

Field	Detail
Issue	Running two PdfConverter instances concurrently (page-batch workers sharing one process) kills the surya llama-server with bad_variant_access was thrown in -fno-exceptions mode. It is a native abort, so Python emits no traceback — the process exits with 0x80000003 and any unflushed progress output is lost.
Found Date	2026-07-21
Status	Closed — approach abandoned
Severity	HIGH
File	No longer in the tree; the parallel batching code was reverted from codes/marker_test.py (see ADR-008).
Description	Found while implementing parallel 20-page batch conversion. Reproduced deterministically: pages 0–59 of the RFT corpus split into three 20-page batches crashes with max_workers=2 and completes cleanly with max_workers=1 (two consecutive repeats each way). A lighter smoke test — two concurrent 3-page batches — passes, so the trigger is sustained concurrent full-page OCR load, not converter construction or server startup. Sharing one create_model_dict() across workers (rather than building one per worker) was already in place and did not prevent it.
Root Cause	Not isolated; inside llama.cpp, which is built -fno-exceptions so the failed std::get/variant access aborts instead of unwinding. Suspected mishandling of overlapping multimodal requests across parallel server slots.
Solution	Not fixed — the feature that triggered it was removed. Concurrent document-level converters are not used on this hardware; Marker runs one converter sequentially. Recorded so the crash is recognised rather than re-discovered if multi-worker conversion is ever revisited. Note that document-level threading was redundant anyway: surya already runs one llama-server with SURYA_INFERENCE_PARALLEL (default 8) slots and coalesces pages across in-flight requests.
Date Resolved	2026-07-21 (by removal)

BUG-017 · Marker 2.0.0 routed every conversion through Docker and failed with no Docker daemon

Field	Detail
Issue	After upgrading to marker-pdf==2.0.0, every file failed with docker run failed: failed to connect to the docker API at npipe:///pipe/dockerDesktopLinuxEngine. Marker had never previously required Docker.
Found Date	2026-07-21
Status	Closed
Severity	CRITICAL
File	codes/marker_test.py; .venv-marker (surya-ocr 0.22.1)
Description	Marker 2.0.0 depends on surya-ocr 0.22.1, which no longer runs the VLM in-process. It starts a separate inference server speaking OpenAI-compatible chat completions and auto-selects the backend from the hardware: surya/inference/_init_.py returns vllm whenever an NVIDIA GPU is detected, and the vllm backend spawns the vllm/vllm-openai Docker image. With the daemon stopped, all conversions failed before any page was processed. Note the detector falls back to nvidia-smi, so it would have selected vllm even on a CPU-only Torch build — this was not a side effect of restoring CUDA.
Root Cause	An architectural change upstream (in-process models → out-of-process VLM inference server) combined with a hardware-derived backend default that assumes a server-class deployment.
Solution	Force the native llama.cpp backend by setting SURYA_INFERENCE_BACKEND=llamacpp before Marker imports surya.settings, and install the llama-server binary (llama.cpp b10068, CUDA 13.3 build plus the matching cudart runtime) under %LOCALAPPDATA%\llama.cpp. Docker/vllm was rejected as impractical on 8 GB of laptop VRAM (ADR-009). A secondary failure followed: appending the install directory to the user PATH does not reach already-running shells, so llama-server binary not found persisted until restart; marker_test.py now falls back to setting LLAMA_CPP_BINARY to the default install path when shutil.which() comes up empty, so no machine-specific absolute

	path is hard-coded.
Date Resolved	2026-07-21

BUG-018 · llama.cpp rejected surya's guided-layout JSON schema as an unparseable grammar

Field	Detail
Issue	With the llama.cpp backend running, every inference request returned 400 - Failed to initialize samplers: failed to parse grammar, repeated once per page, producing no output.
Found Date	2026-07-21
Status	Closed
Severity	HIGH
File	codes/marker_test.py; .venv-marker (surya-ocr 0.22.1)
Description	surya/layout/_init_.py sends LAYOUT_JSON_SCHEMA as a strict response format for guided decoding. The schema uses an anchored <code>^d{1,4} d{1,4} d{1,4} d{1,4}\$</code> bbox pattern plus integer minimum/maximum bounds. vllm enforces these natively; llama.cpp must first convert JSON Schema to GBNF and fails on that conversion, so the sampler never initializes and the request is rejected outright.
Root Cause	Guided-decoding support is backend-specific. surya's schema is written against vllm's constraint engine, and the llama.cpp fallback path was not exercised with it.
Solution	Set SURYA_GUIDED_LAYOUT=false before import. surya already ships the sibling setting SURYA_GUIDED_TABLE_REC disabled by default with the note that the model emits well-formed JSON without the schema, so the same reasoning applies. The trade-off is that layout output is no longer schema-constrained, which permits longer and occasionally malformed generations.
Date Resolved	2026-07-21

BUG-019 · Orphaned torchaudio broke the surya OCR-error server after the CUDA Torch swap

Field	Detail
Issue	Conversions failed with <code>ocr_error</code> server failed to become healthy at <code>http://127.0.0.1:...</code> within 300.0s. The real error appeared only in the captured child-process log: <code>OSError: Could not load this library: ...\\torchaudio\\lib\\libtorchaudio.pyd</code> .
Found Date	2026-07-21
Status	Closed
Severity	HIGH
File	<code>.venv-marker</code> (torchaudio 2.11.0+cu128)
Description	Reinstalling Torch as <code>2.13.0+cu130</code> left a stale <code>torchaudio 2.11.0+cu128</code> whose compiled extension is ABI-bound to Torch 2.11 and can no longer load. <code>transformers/loss/loss_rnnt.py</code> guards its import with <code>is_torchaudio_available()</code> , which only checks whether the package is <i>installed</i> , not whether it <i>loads</i> — so the guard passed and the import died at the DLL level. That import chain is reached by <code>transformers.models.distilbert</code> , which the surya <code>ocr_error</code> predictor loads, killing the server subprocess. Because the failure happened in a child process, no Python traceback surfaced in the main run — only an opaque 300-second health-check timeout.
Root Cause	A dangling companion package left behind by an intentional Torch build swap, combined with a <code>transformers</code> availability check that tests installation rather than importability.
Solution	<code>uv pip uninstall torchaudio</code> . Nothing in Marker or surya uses it — only <code>transformers</code> ' unrelated audio models — and no <code>torchaudio==2.13.0</code> exists on the <code>cu130</code> index to install instead. With the package gone, <code>is_torchaudio_available()</code> returns <code>False</code> and the import is correctly skipped. Recorded as a general hazard: after any Torch reinstall, audit companion packages (<code>torchvision</code> , <code>torchaudio</code>) for ABI drift, and treat "server failed to become healthy" as a signal to read the captured server log rather than the console.
Date Resolved	2026-07-21

BUG-020 · Markdown table renderer exceeded maximum recursion depth on a degenerate VLM-reconstructed table

Field	Detail
Issue	A source PDF failed with <code>maximum recursion depth exceeded, immediately preceded by marker: Overflow in columns: 0 >= 3 or rows: 1 >= 1.</code>
Found Date	2026-07-21
Status	Open — not reproduced against a minimal case
Severity	MEDIUM
File	<code>.venv-marker (marker/renderers/markdown.py)</code>
Description	Occurs at render time, not during extraction. The preceding log line originates from the overflow guard in <code>marker/renderers/markdown.py</code> , indicating a table whose declared cell spans are mutually inconsistent (zero columns against three, one row against one). The renderer then recurses over the malformed structure until Python's limit is reached. The same run reported 52 tables of which 39 were OCR-reconstructed by the VLM.
Root Cause	Not isolated; inside Marker's Markdown renderer. Marker 2.0.0 has no dedicated table model — tables come from pdftext heuristics on digital pages and from full-page VLM OCR otherwise — so table structure is now generated text and can be self-inconsistent in ways the 1.10.2 table model did not produce.
Solution	None applied. The failing document is not part of the current two-file corpus, so it does not block work. It is a per-file failure that the batch loop already reports and continues past. Recorded because it is a new failure mode introduced by VLM-generated table structure and belongs with the existing table-fidelity findings (BUG-011).
Date Resolved	Not resolved
Follow-up (2026-07-23)	Root-caused and re-filed as BUG-024. The attribution above — "the renderer then recurses over the malformed structure" — is wrong, and the wrongness came from reading log adjacency as causation. <code>Overflow in columns</code> is emitted at

	marker/renderers/markdown.py:182 inside an <code>except IndexError</code> that logs and continues; it never raises. The <code>Table OCR</code> failed for block <code>/page/28/Table/6</code> line above it comes from a different stage entirely (<code>marker/processors/table.py:169</code>), which also logs and continues — the whole table stage then printed its completion stats successfully, ~3.5 s before the crash. The recursion is in <code>markdownify</code>'s DOM walk over the document's HTML and is driven by nesting depth, not by table validity. This entry stays as written; BUG-024 carries the measured mechanism.
--	---

BUG-021 · preprocessing() promoted numbered clauses to headings shallower than their own section

Field	Detail
Issue	A run of numbered clauses was promoted to a heading level <i>above</i> the section heading containing it — <code>#### **5. INSTRUCTIONS TO BIDDERS**</code> followed by <code>### 5.1, ### 5.2, ...</code> — so every clause parsed as a sibling of its own parent and any consumer that groups by ATX level nested the document inside out.
Found Date	2026-07-22
Status	Closed
Severity	HIGH
File	<code>chunking/ingest.py (apply_heading_branch(), promote_italic_sublabels())</code>
Description	Found while tracing why <code>temp_split()</code> tore clause 5.4 apart (BUG-022): the splitter's "deepest ATX level in this window" rule was selecting the section heading itself together with two sub-labels, which is only possible if the levels between them are inverted. <code>apply_heading_branch()</code> chose the promoted level from whichever block in the sequence Marker had <i>already</i> written as a heading. In this document Marker emitted <code>### 5.6</code> and <code>### 5.7</code> while leaving 5.1–5.5 as plain text, so the whole sequence inherited level 3 — one level shallower than the enclosing <code>####</code> section. <code>promote_italic_sublabels()</code> then compounded it by deriving <code>#### Technical Bids:</code> from <code>last_level + 1</code>, placing a sub-label at the section's own level.

Root Cause	The promoted level was read only from the sequence itself, with no reference to the heading hierarchy the sequence sits inside, so Marker's own inconsistent heading levels propagated straight into the normalized output.
Solution	Added <code>_enclosing_heading_level()</code> and treated any existing prefix as a <i>floor</i> : the promoted level is raised to at least the nearest preceding heading's level, clamped to 6. Level-with-the-parent rather than one below it was chosen on evidence, not intuition — diffing <code>preprocessing(dataset/input.md)</code> against <code>dataset/expected-output.md</code> showed <code>enclosing + 1</code> deviating on 84 heading lines and 604 diff lines overall, while level-with-parent reproduced the ground truth's levels exactly (400 diff lines, the remainder a block move rather than level changes). Level-with-parent is sufficient because a section heading left owning nothing is never stranded: the packer carries a headings-only buffer forward into the clause that follows it.
Date Resolved	2026-07-22

BUG-022 · `temp_split()` cut at the deepest structural boundary instead of the last one that fit

Field	Detail
Issue	Two complementary failures from one rule: chunks far below <code>CHUNK_SIZE</code> sitting next to content that would have fitted (a 64-character opening chunk beside ~1,500 characters of the same cover page), and whole sections torn in half at a subsection boundary (clause 5.4 split between its <code>Technical Bids</code> and <code>Commercial Bids</code> sub-labels).
Found Date	2026-07-22
Status	Closed — algorithm replaced
Severity	HIGH
File	<code>chunking/ingest.py</code> (<code>_structural_cut()</code> and its supporting helpers, all now removed)
Description	Reported by the user from a chunk-run report and reproduced by instrumenting <code>_structural_cut()</code> . Its heading rule located the <i>deepest</i> ATX level present in the candidate window and pushed

	that run of subsections forward. At offset 0 the deepest level was the contents heading, so the cut landed there and <code>enclosing_heading_start()</code> then pulled it back to the <code>#</code> title, stranding 64 characters. At section 5 the inverted levels from BUG-021 made "deepest level 4" mean <i>the section heading plus the two bid sub-labels</i>, so the cut landed on <code>Commercial Bids</code> and split clause 5.4. Three sibling helpers (<code>list_section_after_heading()</code>, <code>table_section_after_heading()</code>, <code>list_section_after_paragraph()</code>) worked against packing from the other side, emitting a matched section as a chunk of its own and never filling it further.
Root Cause	Deepest-ATX-level is a proxy for section ownership that only holds when heading levels are well-formed, which Marker's are not (BUG-021). More fundamentally the algorithm selected a boundary against a size limit rather than filling a chunk, so nothing in it could express "take whole units until the next will not fit".
Solution	Rewrote <code>temp_split()</code> as parse-then-pack rather than patching a seventh rule into a set whose rules already contradicted one another — <code>list_section_after_heading()</code> deliberately emitted under-full chunks, so a global minimum-fill rule would have needed carve-outs for its own siblings (ADR-010). <code>structural_cut()</code> and eleven helpers that existed only to serve it were deleted; <code>ingest.py</code> went from 1,176 to 974 lines with no unreferenced functions left. Verified by whitespace-normalised round trip across all nine local Markdown files: zero loss, opening chunk 780 characters through the contents list, clause 5.4 whole with 5.5 packed in behind it.
Date Resolved	2026-07-22

BUG-023 · `_pack()` closed chunks on a trailing heading, labelling the wrong content

Field	Detail
Issue	A chunk could end on a heading whose content began the next chunk — e.g. one chunk closing <code>#### **APPENDIX 1**</code> while <code>#### **BID TIMETABLE**</code> and its table opened the following one. A heading describes what comes after it, so publishing it at a chunk's end labels the wrong content and leaves the real content unlabelled. 43 of 1,042 chunks were affected at <code>CHUNK_SIZE=2400</code>.
Found Date	2026-07-22
Status	Closed
Severity	MEDIUM
File	<code>chunking/ingest.py</code> (<code>_pack()</code> , <code>split_node()</code> , <code>trailing_heading_start()</code>)
Description	Reported by the user from a chunk-run report; measured across the corpus before fixing, which showed it was a class rather than an instance and revealed two distinct paths. Most cases were a heading-only unit that is a <i>sibling</i> of the section it introduces (<code>APPENDIX 1</code> and <code>BID TIMETABLE</code> are both <code>####</code>), so it became its own unit and was buffered onto the tail of the previous chunk. Fixing that left five, all sharing a different shape: a heading-only unit that is the <i>last child</i> of a section, flushed by the inner <code>_pack()</code> after its loop while the outer document still had text. The tell was that every survivor was followed by a shallower heading.
Root Cause	A regression introduced by the ADR-010 rewrite. The predecessor algorithm carried a <code>trailing_heading_start()</code> guard; the rewrite dropped it and substituted a check for a buffer holding <i>only</i> headings, which does not catch a buffer that merely <i>ends</i> in one. The inner-loop path additionally had no way to know whether the caller still had text after it.
Solution	Restored <code>trailing_heading_start()</code> in <code>_pack()</code>'s flush, and gave <code>_pack()/split_node()</code> a <code>tail_open</code> flag so a nested pack returns its trailing heading run instead of emitting it, letting the caller place it in front of what it introduces. When the next node is a table,

	the carried heading becomes part of the prefix repeated on every table piece (ADR-011). Verified at 0 of 1,041 chunks; the document's final chunk is exempt, since nothing follows it to mislabel.
Date Resolved	2026-07-22
Follow-up (2026-07-22)	A second shape of the same defect: a chunk closing on a heading <i>plus</i> the <code></code> anchors trailing it, e.g. ... <code> / ##### **5. VERIFICATION OF COMPANY INFORMATION** / </code>. The first fix walked back over blank lines only, so an anchor after the heading stopped the walk and the heading stayed put. <code>trailing_heading_start()</code> now also walks over markup-only lines (<code>is_markup_line()</code>) while still requiring a heading in the run — an anchor annotates whatever follows it, so one sitting after the heading travels with it, and one sitting <i>before</i> the heading belongs to the body already recorded and stays. A tail of markup with <i>no</i> heading in it is deliberately not moved: that broader rule ("never end a chunk on any HTML tag") was implemented, measured at 19 markup-only chunks and 125 chunks ending on markup, and rejected by the user as wrong, then fully reverted. <code>contains_only_headings()</code> was widened the same way through a shared <code>has_body()</code>, so a heading accompanied only by anchors is still recognised as a body-less buffer and carried forward whole rather than split into an anchor-only chunk.

BUG-024 · Marker's Markdown renderer recursed past Python's limit on deeply nested HTML

Field	Detail
Issue	A source PDF failed with a bare <code>RecursionError: maximum recursion depth exceeded</code> and no location. Re-filed from BUG-020 after the crash was located in <code>markdownify</code> 's DOM walk rather than in table rendering.
Found Date	2026-07-21 (as BUG-020); root-caused 2026-07-23
Status	Open — mechanism measured, fix implemented and then removed by the 1.10.2 revert (ADR-012)
Severity	HIGH

File	<code>.venv-marker (marker/renderers/markdown.py, markdownify/_init_.py)</code> ; the removed instrumentation lived in <code>codes/marker_test.py</code> at <code>b73c2e3</code>
Description	<code>MarkdownRenderer</code> hands the assembled document HTML to <code>markdownify</code>, which parses it with Python's <code>html.parser</code> and walks the tree through mutually recursive <code>process_tag()</code> / <code>process_element()</code>. <code>html.parser</code> does not auto-close unclosed inline tags, so each unclosed <code><i>//</code> makes the rest of its block a child of itself. Measured directly: N unclosed <code><i></code> tags inside one <code><p></code> produce a DOM of depth N+1, and <code>markdownify</code> burns ~1.99 Python frames per DOM level, so the default limit of 1000 tops out around 480 levels — depth 481 converted, depth 501 raised. The depth is bounded rather than infinite: DOM depths of 1001, 3001 and 8001 all converted at limit 20000.
Root Cause	Malformed inline markup in the VLM-generated HTML for a single block, combined with a non-auto-closing HTML parser and a recursive serializer. Not table-specific — a deeply nested list or a run of unclosed emphasis produces the same shape.
Solution	Implemented at <code>b73c2e3</code> and reverted at <code>aa1627f</code>: <code>run_with_deep_stack()</code> ran the conversion on a spawned thread with <code>sys.setrecursionlimit(20000)</code> and a 128 MB stack, plus a <code>recursion_report()</code> that printed the repeating frame cycle and depth-at-failure. Raising the limit alone was rejected as actively harmful — the limit caps Python's frame counter but does not grow the 1 MB Windows main-thread stack the default is sized around, so it trades a catchable <code>RecursionError</code> for a silent process death; <code>threading.stack_size()</code> applies only to newly started threads, hence the wrapper. <code>threading.stack_size()</code> also raises <code>ValueError</code> rather than clamping at ≥ 256 MB (<code>THREAD_MAX_STACK_SIZE</code>), so an initial 512 MB default would have aborted the script on every run; 255 MB is the measured ceiling. Splitting the PDF was measured and ruled out as a workaround (see BUG-025's note and Research topic 12): a closed block-level tag resets nesting, so depth is set by the single worst block and is unchanged by document length or <code>page_range</code>. The fix is not in the tree. <code>markdownify</code> with <code>html.parser</code> is the renderer path in the reverted <code>marker-pdf==1.10.2</code> as well, so the failure mode survives the revert while the instrumentation does not.
Date Resolved	Not resolved

BUG-025 · One dead pdftext worker aborts the whole PDF text extraction

Field	Detail
Issue	A source PDF failed with A worker process died while extracting, during PDF text extraction — before any GPU stage ran.
Found Date	2026-07-22 (symptom); root-caused 2026-07-23
Status	Open — root cause identified, fix implemented and then removed by the 1.10.2 revert (ADR-012)
Severity	HIGH
File	.venv-marker (pdftext/extraction.py, marker/providers/pdf.py); the removed fix lived in codes/marker_test.py at b73c2e3
Description	pdftext sizes its pool as <code>min(workers, len(page_range) // WORKER_PAGE_THRESHOLD)</code> with <code>WORKER_PAGE_THRESHOLD = 10</code> , and only bypasses the <code>ProcessPoolExecutor</code> entirely when that lands ≤ 1 . Marker's PDF provider defaults <code>pdftext_workers</code> to 4, so every document in this corpus (154, 154 and 192 pages) spawned 4 workers, and one dying took the whole conversion down with it. Corroborating evidence upstream: Marker's own CLI, server, and Streamlit front-ends (<code>marker/scripts/parser.py:105</code> , <code>server.py:94</code> , <code>streamlit_app.py:31</code>) all force <code>config["pdftext_workers"] = 1</code> themselves, while <code>PdfConverter</code> 's programmatic default does not.
Root Cause	A multi-process fan-out with no per-worker recovery, enabled by default at a page count every document here exceeds. The individual worker's death was not isolated; the parent only ever sees the pool's aggregate failure.
Solution	Implemented at b73c2e3 and reverted at aa1627f: <code>config={"pdftext_workers": 1}</code> on the converter, which makes the pool-size expression land at 1 and skips the <code>ProcessPoolExecutor</code> altogether. Cheap insurance rather than a real slowdown — extraction is pure PDF text parsing, seconds beside the GPU stages. Page batching was explicitly evaluated as an alternative and rejected on arithmetic: a 50-page slice still yields <code>min(4, 5) = 4</code> workers, so it does not avoid the pool at all. The fix is not in the tree, and the reverted pdftext 0.6.3 has the

	identical pool-sizing expression and the same <code>pdftext_workers = 4</code> provider default, so the failure mode survives the revert.
Date Resolved	Not resolved

BUG-026 · Marker 2.0.0 balanced collapsed a 154-page PDF into one 215-column table

Field	Detail
Issue	<code>marker_results/src2.md</code> is 50,084,774 characters across 244 lines — a 50 MB Markdown file from a 9.4 MB, 154-page source. 99.99% of it is pipe-table rows.
Found Date	2026-07-23
Status	Open — the output is in the tree and has not been regenerated
Severity	CRITICAL
File	<code>marker_results/src2.md</code> ; <code>.venv-marker</code> at the time (<code>marker-pdf 2.0.0</code> , <code>balanced</code> mode)
Description	Lines 162–244 are a single table of 83 rows × 215 columns = 17,845 cells, of which only 394 hold any content. Marker pads every cell to its column's widest, so each of those 83 rows is 603,330 characters wide and 98.8% of the file is padding whitespace. The document's text is largely present — 105,446 alphabetic tokens — but its structure is gone: 9 headings across 154 pages, against 242 in <code>src1.md</code> and 207 in <code>src3.md</code> from the same run. The other two files of that run are unremarkable, with widest table rows of 1,151 and 1,305 characters, and the July 22 RFT volume's widest row is 723. This is a single-document blowup, not a corpus-wide regression.
Root Cause	Not isolated. Consistent with the absence of a dedicated table model in 2.0.0 (BUG-020's root cause): table structure comes from full-page VLM OCR, so a mis-segmented page can declare an arbitrary column count and absorb subsequent content into the same grid. The 250-page RFT volumes converted under the same pipeline on 2026-07-22 without this.
Solution	None applied. It is the concrete evidence behind the decision to abandon 2.0.0 and revert to 1.10.2 (ADR-012; Research topic

	13). Two downstream consequences are recorded rather than fixed: the file is inside the chunking corpus, so <code>chunking/ingest.py</code> will attempt <code>split_table()</code> (ADR-011) on a 603,330-character single row — Research topic 11 recorded 897 characters as the corpus maximum and treated the oversized-row and oversized-cell rules as unreachable, which this file makes false — and the tree has not been re-chunked since, so that interaction is predicted, not observed.
Date Resolved	Not resolved

BUG-027 · ripgrep silently returns no matches inside the loader virtual environments

Field	Detail
Issue	Searches for known-present strings in the installed Marker/surya source returned "No matches found". The text was there; the tool had not read the files.
Found Date	2026-07-23
Status	Closed — working practice, no code change
Severity	MEDIUM
File	<code>.gitignore</code> ; any search across <code>.venv-marker/</code> , <code>.venv-docling/</code> , <code>.venv-unstructured/</code>
Description	The three loader environments are Git-ignored, and ripgrep honours <code>.gitignore</code> by default, so it skips them without saying so. Several searches during the 2026-07-23 investigation came back empty and were briefly read as evidence that a log string did not exist in the installed tree. Caught by grepping for a string known to be present and getting the same empty result.
Root Cause	A silent negative: the tool reports "no matches" identically whether a file was searched and did not match, or was never searched at all.
Solution	Use PowerShell <code>Select-String (Get-ChildItem -Recurse Select-String)</code> for anything under a <code>.venv-*</code> directory, or pass ripgrep <code>--no-ignore/-u</code> . General practice recorded alongside BUG-004: before trusting a negative search result, confirm the tool can find

	something you know is there. Because the loaders' behaviour lives almost entirely in Git-ignored installed packages, this will recur in every investigation of this repository.
Date Resolved	2026-07-23